



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 · Design & Analysis of Algorithms · Fall 2026

Cases, and Two-Sum Three Ways

Which inputs you analyse, which bound you claim, and how an exponent falls

Session 6 of 24 · Tuesday 6 October 2026 · 2:30 – 3:50 PM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 6 OF 24

Cases, and Two-Sum Three Ways

Tuesday, 6 October 2026 · 2:30 – 3:50 PM

LEARNING OBJECTIVES

- ✓ Analyse the best, worst and average case of an algorithm separately.
- ✓ Explain why the *case* analysed and the *bound* claimed are independent choices.
- ✓ Justify why worst case is the default in this course.
- ✓ Take one problem from $\Theta(n^2)$ to $\Theta(n \log n)$ to $\Theta(n)$ and explain what changed each time.
- ✓ Report auxiliary space alongside time.

FROM SESSION 5

We can count a loop nest. But `linear_search` has no fixed count at all: on one input it does 1 comparison and on another it does n . Both inputs have the same size. Counting alone cannot resolve that, and today's first idea does.

1 · Three Cases 18 min

Two inputs of the same size need not cost the same. So before giving a number we must say **which inputs we are talking about**. Take linear search on a list of n elements:

Case	Which input	Comparisons	Bound
Best	target sits at index 0	1	$\Theta(1)$
Worst	target is last, or absent	n	$\Theta(n)$
Average	target present, position uniform	$(n+1)/2$	$\Theta(n)$

The average deserves a moment. If the target is equally likely to be at any of the n positions, the expected number of comparisons is

$$(1 + 2 + \dots + n) / n = n(n+1)/2n = (n+1)/2$$

— the arithmetic series from Session 5 again. Note that the average is $\Theta(n)$, the *same class* as the worst case. Halving the work did not change the class, exactly as the triangle did not.

Average case hides an assumption

“Uniformly distributed” is a claim about your data, not a mathematical fact. If your users search for popular items far more often than rare ones, the real average is much better than $(n+1)/2$. If the item is usually absent, it is worse. Whenever you quote an average case, you owe your reader the distribution you assumed.

2 · Case and Bound Are Different Questions 15 min

This section exists to kill one specific and very durable error.

The error

“O is the worst case, Ω is the best case, Θ is the average case.”

This is **false**, and students who believe it get the right answer often enough that nobody corrects them until an exam does.

The truth

They are two independent axes. The **case** chooses which inputs you look at. The **bound** describes how the cost of *those* inputs grows.

	O (ceiling)	Ω (floor)	Θ (exact)
Best case	best case is $O(1)$	best case is $\Omega(1)$	best case is $\Theta(1)$
Worst case	worst case is $O(n)$	worst case is $\Omega(n)$	worst case is $\Theta(n)$
Average case	average is $O(n)$	average is $\Omega(n)$	average is $\Theta(n)$

Every one of those nine statements is meaningful, and for linear search every one of them is true. The grid is filled in independently: pick a row, then pick a column.

WHY WORST CASE IS OUR DEFAULT

It is the only one that is a **promise** rather than a hope. The best case tells you what happens on a lucky day. The average tells you what happens if your assumption about the data is right. The worst case tells you what happens on the day the data is unkind — which, for anything that runs in production, is the day that decides whether your system stays up. Unless stated otherwise, every complexity claim in this course is worst case.

3 • Case Study: Two-Sum 30 min

Problem. Given a list of integers `A` and a target `T`, return indices $i \neq j$ with `A[i] + A[j] == T`, or `None` if no such pair exists.

3.1 Attempt 1 — try every pair

PY BRUTE FORCE

```
def two_sum_quadratic(A, T):
    """Correct, obvious, and quadratic."""
    for i in range(len(A)):
        for j in range(i + 1, len(A)):
            if A[i] + A[j] == T:
                return (i, j)
    return None
```

$\Theta(n^2)$ worst case

The dependent nest of Session 5: $n(n-1)/2$ pairs examined when no answer exists. Best case $\Theta(1)$ — the answer is the first pair tried.

Before improving it, ask what the inner loop is actually *doing*. For a fixed `i`, it asks: **does the value `T - A[i]` appear elsewhere in the list?** That is a search problem, and we spent four sessions on search.

THE REFRAMING IS THE WHOLE TRICK

We did not start looking for a cleverer way to examine pairs. We noticed the inner loop was a *search*, and we already know two ways to make searching fast: sort the data, or index it. Each gives an algorithm.

3.2 Attempt 2 — sort, then squeeze from both ends

PY SORT AND SQUEEZE

```
def two_sum_sorted(A, T):
    """Sort once, then walk inward from both ends."""
    order = sorted(range(len(A)), key=lambda i: A[i]) # O(n log n)
    lo, hi = 0, len(A) - 1
    while lo < hi: # O(n) in total
        s = A[order[lo]] + A[order[hi]]
        if s == T:
            return (order[lo], order[hi])
        if s < T:
            lo += 1 # sum too small: the smallest value cannot help, discard
        else:
            hi -= 1 # sum too large: the largest value cannot help, discard
    return None
```

Why is the sweep correct? When the sum is too small, the current smallest value paired with the *largest* remaining value was its best chance, and it failed — so that value can never be part of an answer and is discarded for ever. The mirrored argument handles the too-large case. Each step discards one value permanently, so the sweep makes at most n steps.

$\Theta(n \log n)$ The sort dominates the linear sweep, so $\Theta(n \log n) + \Theta(n)$ collapses to $\Theta(n \log n)$. Auxiliary space $\Theta(n)$ for the index ordering.

3.3 Attempt 3 — remember what you have already seen

PY ONE PASS WITH A DICTIONARY

```
def two_sum_hash(A, T):
    """One pass, one dictionary."""
    seen = {} # value -> index where we saw it
    for i, v in enumerate(A):
        if T - v in seen: # O(1) average lookup
            return (seen[T - v], i)
        seen[v] = i
    return None
```

Read the loop carefully: at the moment we reach element `v`, the dictionary holds every element to its left. So asking whether `T - v` is in `seen` asks exactly “has the partner of `v` already gone past?” Every pair is checked exactly once, when its *second* member is reached.

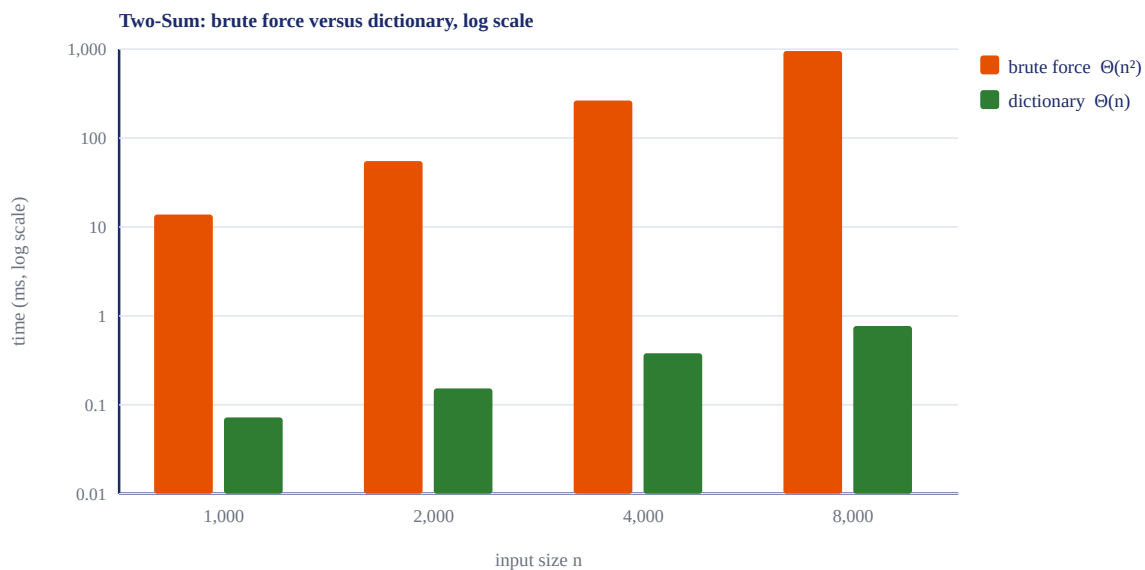
$\Theta(n)$ average One pass, constant expected work per element, $\Theta(n)$ auxiliary space. The inner search did not get faster — **it disappeared**.

3.4 The measurement

MEASURED ON THE LECTURE MACHINE

TWO SUM (worst case: no pair exists)

n	quadratic (ms)	dictionary (ms)	ratio
1,000	13.8	0.072	192x
2,000	55.2	0.153	361x
4,000	264.1	0.380	695x
8,000	951.6	0.769	1,237x



The doubling test on real data. The orange bars roughly quadruple at each doubling of n ; the green bars roughly double. Note the ratio column above: the advantage is not a fixed factor, it grows — $192\times$ at $n = 1000$ and $1237\times$ at $n = 8000$. A constant-factor improvement would have kept that ratio flat. This is what changing the exponent looks like.

THE THREE MOVES YOU JUST WATCHED

All three attempts are correct. They differ in what each one **remembers**. Brute force remembers nothing and re-derives everything. Sorting imposes an order once and exploits it n times. The dictionary trades memory for time directly. Almost every improvement in the rest of this course is one of these three in disguise.

4 • Space, and the Honest Report 12 min

Auxiliary space is the extra memory an algorithm uses beyond its input. Rank our three attempts by that measure and the order reverses:

Algorithm	Time (worst case)	Auxiliary space
two_sum_quadratic	$\Theta(n^2)$	$\Theta(1)$
two_sum_sorted	$\Theta(n \log n)$	$\Theta(n)$
two_sum_hash	$\Theta(n)$	$\Theta(n)$

On a microcontroller with a few kilobytes of RAM, the “worst” algorithm may be the only one that fits. If the list is already sorted when it arrives, attempt 2 becomes $\Theta(n)$ time and $\Theta(1)$ space and beats everything. **Context decides; analysis informs.**

What a complete answer looks like

“ $\Theta(n)$ worst-case time and $\Theta(n)$ auxiliary space, assuming average-case dictionary behaviour.” Time alone is half an answer, and an unstated assumption is a trap for whoever maintains your code.

SESSION 6 TAKEAWAY

Say which inputs you mean before you say how fast it is. Case and bound are independent choices, and worst case is our default because it is the only promise. When an inner loop is really a **search**, replacing it with an ordering or a dictionary is usually where the exponent falls — and the payment is in memory.

IN-CLASS EXERCISES (15 min — work in pairs)

1. Give the best, worst and average case of binary search on a sorted list of n elements. Which is the odd one out, and why?
2. True or false, with a reason: (a) the best case of an algorithm can be $\Omega(n^2)$; (b) if the worst case is $\Theta(n)$, the average must be $\Theta(n)$; (c) if the average is $\Theta(n)$, the worst must be $\Theta(n)$.
3. Someone reports that their algorithm is “ $O(n)$ in the best case”. What have they actually told you about its performance in production? Is the statement useful?
4. Give an algorithm for Two-Sum that runs in $\Theta(n)$ time and $\Theta(1)$ auxiliary space when the input is *already sorted*.
5. Modify `two_sum_hash` to return *all* pairs summing to T rather than the first. What is the new worst-case time, and what input achieves it?

Homework · due Friday 9 October (Workshop I)

1. Implement all three Two-Sum versions. Verify they agree on 500 random inputs that do contain a pair, then time all three at $n = 1000, 2000, 4000, 8000$ on inputs with *no* pair. Submit the table.
2. Repeat the timing on inputs where the answer is the *first* pair examined. Which algorithm's numbers change most? Relate your answer to §1.
3. Write `three_sum(A, T)` by brute force and state its Θ . Then improve it to $\Theta(n^2)$ by fixing one element and using the sorted two-pointer sweep on the rest. Explain in a paragraph why the dictionary trick does not get you to $\Theta(n)$.
4. For linear search, derive the average number of comparisons when the target is *absent* with probability p and uniformly placed otherwise. Sanity-check your formula at $p = 0$ and $p = 1$.
5. **Bring to Workshop I:** your working code for Two-Sum and `has_duplicate`, plus a written list of any question from Sessions 1–6 that you cannot yet answer confidently. The workshop is built around those questions.

Next session · Friday 9 October — Workshop I: No new material. We work problems from Sessions 1–6 together, at the board and at the keyboard, and clear up whatever is still unclear before we start on recursion.